

ZK-STARK

検証者プロセスの詳細図解

「何を受け取り、何を再計算し、どの不正を拒否するのか」を一工程ずつ追う

検証者が行うのは「計算の再実行」ではなく「証明構造の再構成」である



witness、実行トレース全体、LDE表全体は受け取らない

検証者は少数の開示値を、事前commitment・AIR・OODS/DEEP・FRIへ同時に照合する

図 0 ZK-STARK 検証者の因果連鎖（筆者作成）

2026 年 8 月 補強資料

本資料の目的と読み方

本資料は、ZK-STARK の証明を受け取った検証者が、何を入力として受け取り、どの値を自ら再計算し、どの順序で commitment、AIR、OODS/DEEP、Merkle opening および FRI を照合して、最終的に受理または拒否するのかを、検証者の視点だけに絞って説明するものである。前資料では Reed-Solomon 符号化と検証全体を扱ったが、本資料では proof object の構造、Fiat-Shamir transcript の再生、OODS 点での制約再構成、Merkle 認証経路、FRI 各層の folding 検査、異常系までをさらに細分化する。

最重要の見取り図 検証者は、証明者の計算を最初から再実行するのではない。公開 statement と承認済み設定から検査条件を固定し、proof object に含まれる少数の開示値が、事前 commitment、計算規則、領域外評価、低次数性のすべてに同時に整合するかを再構成する。

対象読者と前提

ZKP 共通の完全性・健全性・ゼロ知識性の一般論は省略する。AIR、LDE、Merkle 木、OODS/DEEP、FRI については、検証者の処理を理解するために必要な範囲で再説明する。本文の数式は古典的な単変数 STARK および二分 FRI を中心に記述し、現代的な S-two / Circle FRI との対応は実装対応章で明示する。

章構成

章	内容
第 1 章	検証者の役割、入力境界、受理の意味
第 2 章	proof object と検証者が外部から固定すべき設定
第 3 章	エンドツーエンドの検証手順を 12 ステップで整理
第 4 章	Fiat-Shamir transcript の再構築
第 5 章	OODS 点における AIR・composition の整合性検査
第 6 章	DEEP / PCS opening による多項式間の結合
第 7 章	query 生成と Merkle decommitment 検証
第 8 章	FRI commit phase・query phase・最終次数検査
第 9 章	受理判定、エラー分類、少数 query でよい理由

章	内容
第 10 章	実装・デプロイ・S-two verifier との対応
Appendix A	F_17 Toy Computation を用いた検証者の数値追跡
Appendix B	実装レビュー用チェックリスト・用語集・参考文献

第 1 章 検証者は何をしているのか

ZK-STARK の検証者は、証明者が実行したプログラムを同じ秘密入力で再実行しない。そもそも検証者は witness を受け取らず、実行トレース全体も受け取らない。検証者が行うのは、公開 statement、承認済み AIR または program ID、証明パラメータ、proof object から、証明者が従ったはずの検査条件を再構築し、proof object の各部分はその条件を満たすかを確認することである。

検証者の入力境界と、検証後に分かること

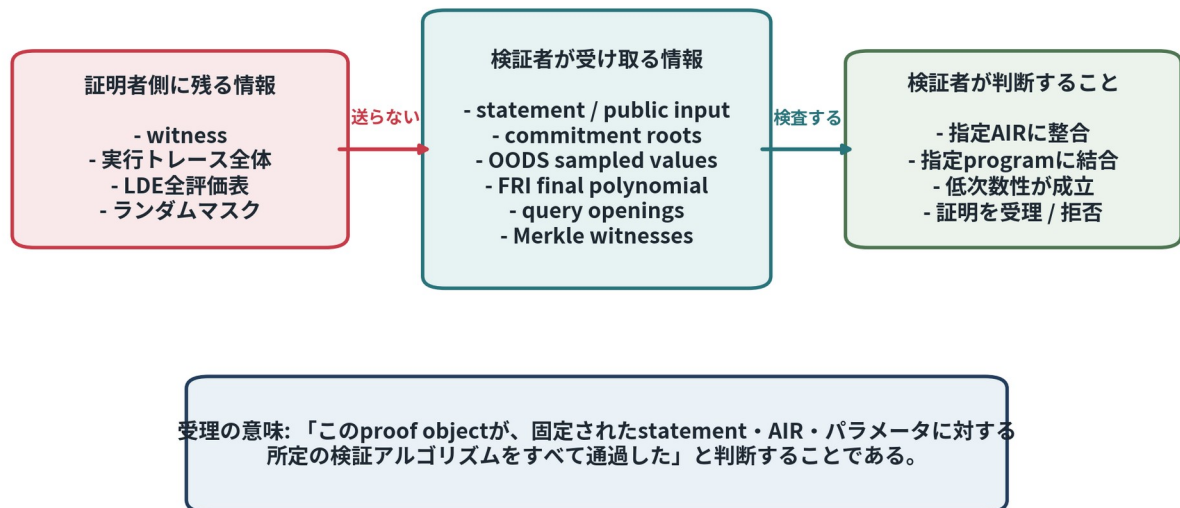


図 1 検証者の入力境界と検証後に分かること（筆者作成）

1.1 受理が意味すること

検証者が Valid を返すとき、それは「公開された program または AIR の制約を満たし、指定された公開入力へ結び付いた低次数証明構造が、設定された soundness parameter の下で検査を通過した」と判断したことを意味する。現実世界のデータが真実であること、アプリケーションの業務要件が正しく AIR へ実装されたこと、証明者端末が侵害されていないことまでを自動的に保証するわけではない。

1.2 者が確認しないもの

- witness の具体的な値や、秘密の実行トレース全体を復元すること。
- 証明者のプログラムを同じ入力で逐次再実行すること。

- 外部データが現実世界で真正であること。外部署名や oracle の検証は AIR 内またはアプリ層で別途必要である。
- 開発者の意図と AIR が一致すること。制約漏れはコード監査・形式検証の対象である。

1.3 検証者の三つの出力

出力	意味	運用上の扱い
Valid	暗号学的検査がすべて成功	アプリ方針をさらに適用して処理を許可する
Invalid	証明内の少なくとも一つの整合性検査が失敗	取引・アクセスを拒否する
Malformed / Unsupported	形式、version、parameter が未対応または不正	暗号検査へ進まず早期拒否する

第 2 章 Proof object と固定設定

proof object は、単一の「証明値」ではない。複数の Merkle root、領域外評価値、FRI 各層の commitment、最終多項式、query opening、authentication pathなどを、所定の serialization で格納した構造体である。検証者は各フィールドを別々に信用するのではなく、Fiat-Shamir transcript と代数式によって相互に結び付ける。

検証者が読む proof object の内部構造

A. 識別・形式情報	proof version / field encoding / column sizes / public input binding
B. Commitments	preprocessed / trace / interaction / composition / FRI layer roots
C. OODS・sampled values	trace mask points の値 / composition OODS evaluation
D. FRI proof	各layer commitment / final polynomial / folding構造
E. Query openings	queried values / Merkle hash witness / column witness
F. 任意のanti-grinding情報	PoW nonce等 - 採用方式・設定に依存

図 2 STARK proof object の主要構成要素（筆者作成）

2.1 検証者が proof 外から固定すべき設定

安全性に直結する設定を proof の自己申告へ任せてはならない。検証者は、承認済み manifest、検証コントラクトの定数、または署名済み設定ファイルから、次の情報を固定する。

分類	固定する代表例	任せると生じる問題
証明対象	program hash、AIR ID、component 構成	攻撃者が常に true を返す別プログラムを証明できる
代数設定	base / extension field、domain、degree bounds	低い次数上限や異なる field で検査を弱められる
FRI 設定	blowup factor、query 数、fold step、last degree	soundness を攻撃者が任意に低下させられる
ハッシュ設定	Merkle hash、transcript hash、domain separator	異なるプロトコル間の transcript 混同が生じる
公開入力形式	順序、長さ、nonce、asset / account binding	別 statement への proof 再利用が生じる
serialization	field の canonical encoding、byte order	同じ意味に複数 byte 表現が生じ、transcript が曖昧になる

2.2 暗号検査より先に行う構造検査

proof parser は、配列長、layer 数、column 数、field element の値域、Merkle witness の上限、final polynomial の係数数などを検査する。これは単なる入力衛生ではない。無制限の配列や異常な log size を受理すると、DoS、整数 overflow、out-of-bounds、transcript 不一致の原因となる。

実装上の原則 「証明として読めるか」を確認してから、「証明が正しいか」を確認する。malformed proof を途中まで暗号処理すると、不要な計算コストと実装攻撃面が増える。

第3章 検証者の処理を 12 ステップで捉える

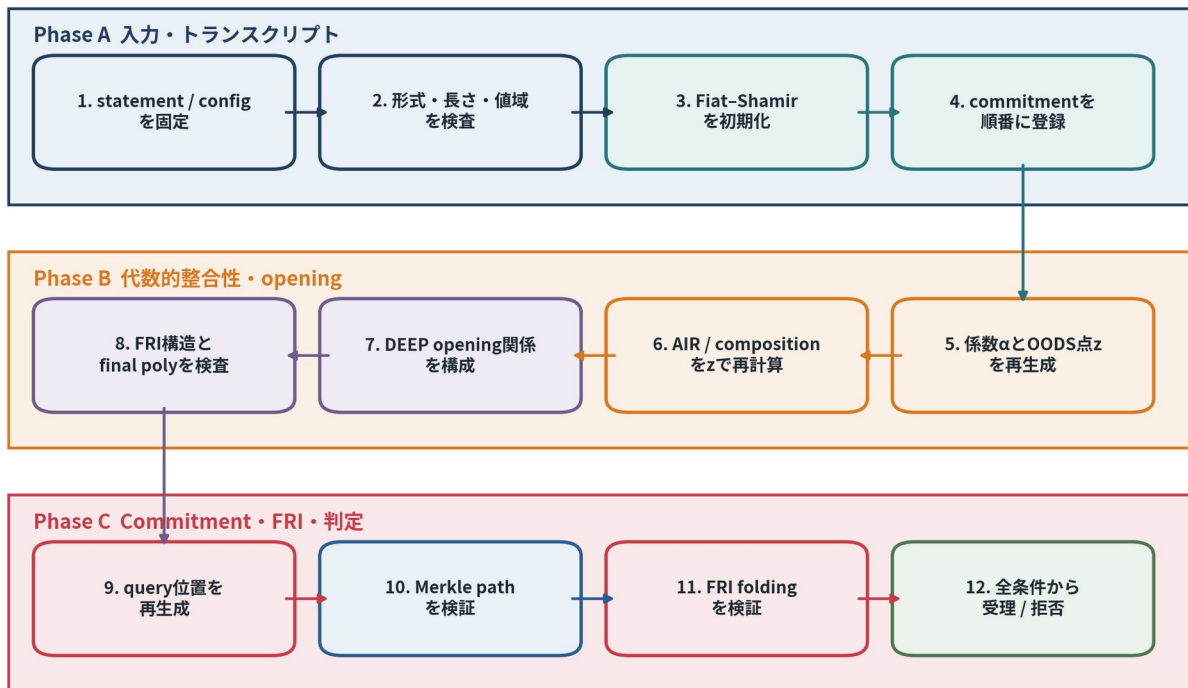


図3 検証者のエンドツーエンド処理 12 ステップ (筆者作成)

ステップ1 検証者は、statement と検証設定を固定する。

公開入力、program / AIR ID、proof version、field、hash、FRI 設定を承認済み manifest から取得する。目的は、攻撃者が検査対象そのものを proof 内で差し替えることを防ぐことである。

ステップ2 検証者は、proof の構造と canonical encoding を査する。

配列長、column log size、field element、final polynomial、Merkle witness を検査する。目的は、曖昧な byte 表現と資源枯渇攻撃を暗号処理の前に拒否することである。

ステップ3 検証者は、Fiat-Shamir channel を初期化する。

protocol tag、version、statement、公開入力、設定 hash を順番どおりに channel へ mix する。目的は、証明を特定プロトコルと特定 statement へ結び付けることである。

ステップ4 検証者は、各 commitment root を所定の順序で登録する。

preprocessed、trace、interaction、composition、FRI layer の root を、仕様に従って channel へ mix する。各 root の直後に必要な challenge を導出する。

ステップ5 検証者は、制約係数と ODS 点を再生成する。

証明者から係数を受け取るのではなく、channel から自ら導出する。目的は、誤りを打ち消す係数や都合のよい評価点を証明者に選ばせないことである。

ステップ 6 検証者は、OODS 点で AIR と composition polynomial を再計算する。

sampled trace values から各制約分子と制約商を求め、ランダム線形結合が proof の composition OODS 値と一致するか確認する。

ステップ 7 検証者は、OODS opening を DEEP / PCS 係へ換する。

各 claim $f_i(z_i)=v_i$ から差分商を構成し、ランダム係数で一つの FRI 対象へ束ねる。目的は、claimed value と commit 済み多項式を結び付けることである。

ステップ 8 検証者は、FRI proof の構造を初期化する。

各 FRI root を mix して folding 係数を導出し、layer 数、degree bound、last layer polynomial の長さを検査する。

ステップ 9 検証者は、全 commitment 後に query 位置を再生成する。

必要なら proof-of-work nonce を確認した後、channel から query index を生成する。目的は、query 位置を証明者が事前選択できないようにすることである。

ステップ 10 検証者は、Merkle authentication path を検証する。

query された値と兄弟 hash から各 root を再構成する。proof 内に余分または不足する witness がないことも確認する。

ステップ 11 検証者は、FRI folding と最終多項式を検証する。

各層の開示値から次層の期待値を再計算し、最後に final polynomial の評価と一致するか確認する。

ステップ 12 検証者は、全検査が成功した場合だけ受理する。

一部の検査を「警告」として無視してはならない。暗号学的 Valid の後で、nonce、失効、version allowlist などのアプリケーション方針を適用する。

第 4 章 Fiat-Shamir transcript を正確に再構築する

非対話型 STARK では、対話型検証者が送るはずのランダム challenge を、それまでの公開 transcript の hash から導出する。検証者は証明者と同一の状態機械を再生しなければならない。値が数学的に同じでも、serialization、順序、長さ prefix、domain separator が異なれば、得られる challenge は変わる。

Fiat-Shamir transcript: commitmentが先、challengeが後

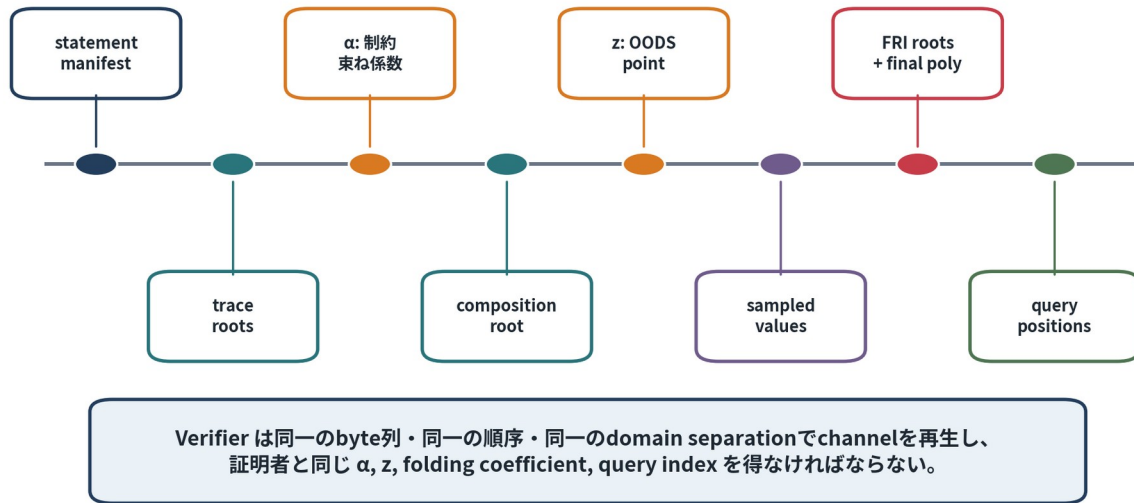


図 4 Fiat-Shamir transcript の時系列 (筆者作成)

$$\text{state}_{\{i+1\}} = H(\text{state}_i \parallel \text{domain_tag}_i \parallel \text{encode}(\text{message}_i))$$

(1)

|| は byte 列連結を表す。

4.1 commitment と challenge の順序

ランダム係数は、係数で束ねる対象が commit された後に導出する。例えば、trace commitment の後に AIR 制約を束ねる係数 α を導出し、composition commitment の後に OODS 点 z を導出する。FRI でも各 layer root を mix した後に、その layer の folding coefficient を導出する。

一つのcommitmentを変更すると、後続の乱数とqueryが連鎖的に変わる

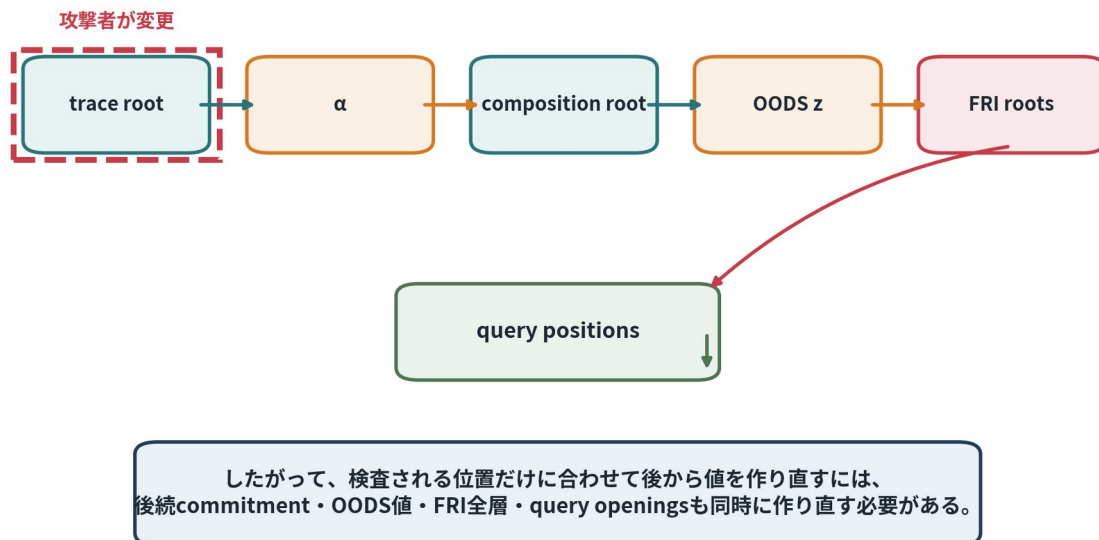


図5 一つの commitment 変更が後続 challenge へ波及する (筆者作成)

4.2 domain separation

同じ hash 関数を使う場合でも、「trace root」「composition root」「FRI root」「public input」などの用途ごとに異なる tag を入れる。これにより、ある種類の byte 列を別種類のメッセージとして解釈する cross-protocol ambiguity を避ける。[注1]

注1 domain separation は、hash 関数の衝突耐性とは別の設計要件である。同じ値を異なる文脈で hash する場合、用途を示す tag と長さ情報を含める。

4.3 field challenge の生成

hash 出力を有限体要素へ写像する方法も仕様の一部である。単純な mod reduction、rejection sampling、複数次 base field 要素から extension field 要素を構成する方法などがある。検証者は同一手順を使い、禁止点や評価領域内の点を引いた場合に再サンプリングする規則も一致させる。

失敗例	発生する現象	対策
byte order が異なる	同じ field element から異なる challenge が出る	canonical little / big endian を仕様化
長さ prefix がない	[a,bc]と[ab,c]の連結が曖昧	固定長または明示長を使用
root の mix 順が異なる	OODS 点以降が全面的に不一致	transcript state machine をテストベクトル化
proof 内設定をそのまま mix	攻撃者が弱い設定を選択可能	承認済み manifest の hash を使用

第5章 OODS 点で AIR と composition を再計算する

OODS 検査では、検証者は通常の LDE 領域外から導出された点 z において、proof が提示したトレース多項式の評価値を使い、AIR 制約と composition polynomial の値を自ら再構成する。ここで確認するのは、単に各値が低次数であることではなく、それらが「同じ計算規則」を表していることである。

OODS点でAIRとcomposition polynomialを再構成する

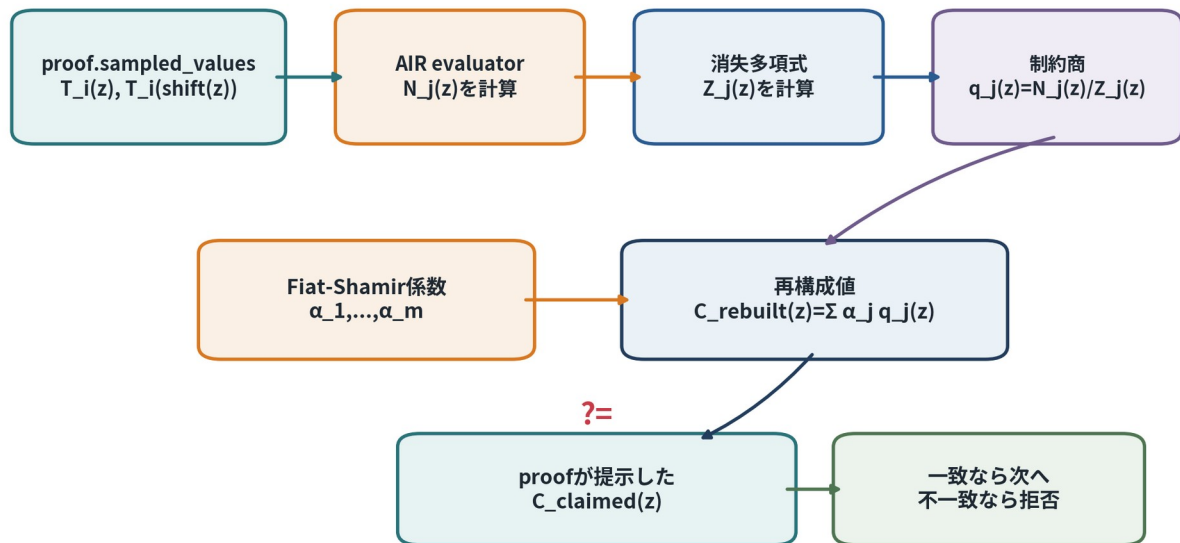


図 6 OODS 点における AIR・composition 再構成 (筆者作成)

$q_j(X) = N_j(X) / Z_j(X)$	(2)
----------------------------	-----

N_j は制約違反を表す分子、 Z_j は制約適用点集合の消失多項式である。

$C(X) = \sum_j \alpha_j \cdot q_j(X)$	(3)
---------------------------------------	-----

α_j は trace 等の commitment 後に Fiat-Shamir から導出する。

$C_{\text{claimed}}(z) \stackrel{?}{=} \sum_j \alpha_j \cdot N_j(z) / Z_j(z)$	(4)
---	-----

検証者が OODS 点で行う中心的な一致検査である。

5.1 mask points を計算する

遷移制約が現在行と次行を使う場合、検証者は z だけでなく、 $\text{shift}(z)$ におけるトレース値も必要とする。AIR component は、各 column についてどの offset の評価が必要かを mask として定義する。検証者は z から必要な sample point 集合を決定し、proof.sampled_values の構造がその集合と一致することを確認する。

5.2 分母が 0 でないことを確認する

制約商 $q_j(z)$ を計算するためには $Z_j(z)$ が 0 であってはならない。OODS 点を評価領域および制約の消失点集合の外から選ぶ理由の一つである。実装は、禁止集合に入った challenge を再サンプリングするか、extension field 上で十分大きな選択空間を確保する。

5.3 OODS 一致だけで十分ではない理由

証明者が OODS 点の値を都合よく作り、commit 済み LDE 表とは無関係な値を送る可能性がある。したがって、OODS sampled values は後続の DEEP quotient と FRI opening proof へ結び付けられなければならない。OODS 一致は計算意味の検査であり、commitment との binding は第 6 章以降が担う。

第 6 章 DEEP / PCS opening で claimed value を commitment へ結ぶ

検証者は、proof が提示した $f(z)=v$ という領域外評価を、単なる自己申告として扱わない。commit 済み評価表が表す低次数多項式 f と claimed value v の差を、 $X-z$ で割った差分商が低次数になることを確認する。これが STARK 型 polynomial commitment の opening の中核である。

OODS opening を DEEP quotient へ埋め込み、commitment と FRI を接続する

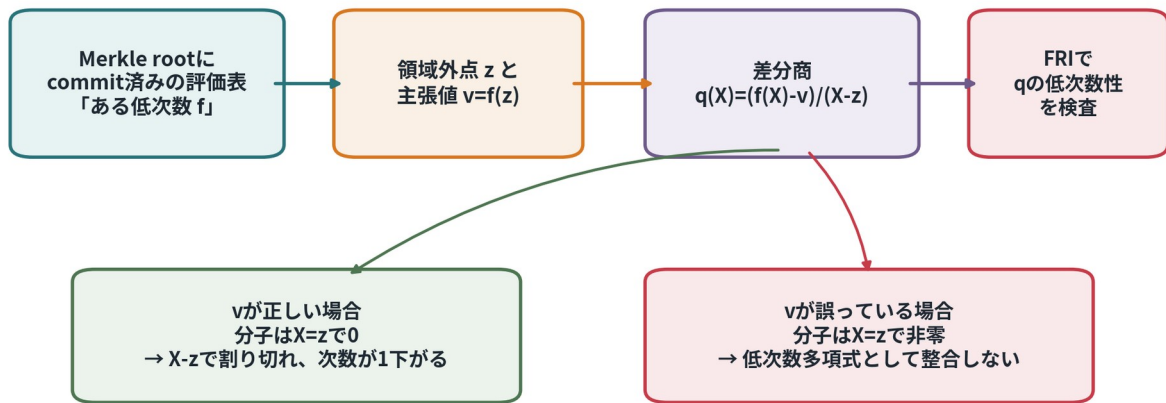


図 7 OODS opening を DEEP quotient へ埋め込む (筆者作成)

$q(X) = (f(X) - v) / (X - z)$	(5)
-------------------------------	-----

$v=f(z)$ が正しければ分子は $X=z$ で 0 となり、 q は多項式になる。

6.1 なぜ誤った v は排除されるのか

v が実際の $f(z)$ と異なる場合、 $f(X)-v$ は $X=z$ で 0 にならず、 $X-z$ で割り切れない。評価領域 D 上では形式的に値を計算できても、それらが所定次数の多項式評価として一貫することは困難になる。FRI は、この差分商またはそのランダム合成が低次数 Reed-Solomon 符号へ近いかを検査する。

6.2 複列複点の batch opening

実用 STARK では、複数の trace column、composition coordinate、複数 offset を同時に open する。各 opening claim の差分商をランダム係数 γ_i で束ね、単一の FRI 対象 P_{DEEP} へ変換する。

degree bound が異なる列を束ねる場合は、必要に応じて次数を正規化する係数や monomial factor を掛ける。

複数の opening claim をランダム線形結合で一つへ束ねる

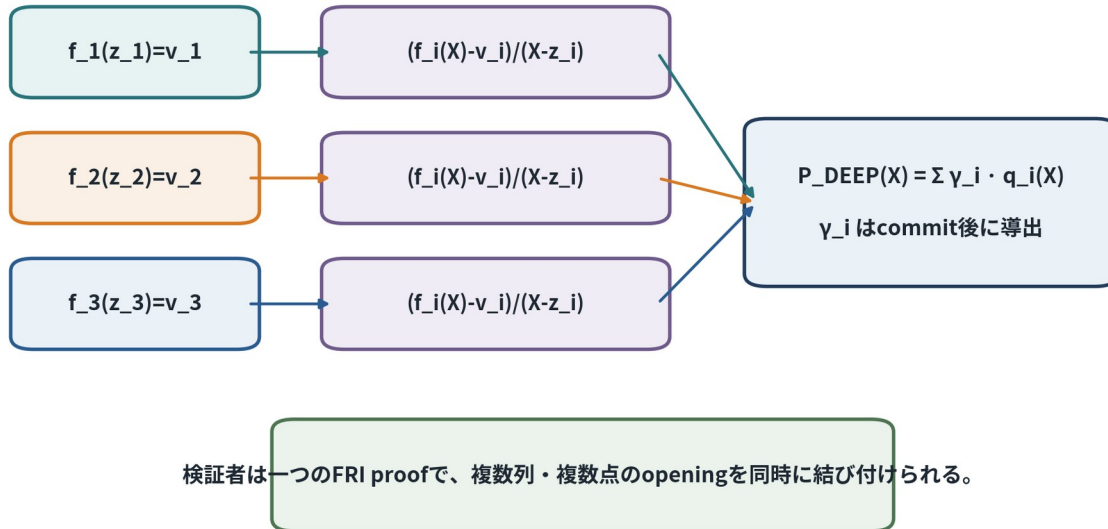


図 8 複数 opening claim のランダム合成（筆者作成）

$P_DEEP(X) = \sum_i \gamma_i \cdot (f_i(X) - v_i) / (X - z_i)$	(6)
---	-----

実装によって次数正規化項や Circle polynomial 用の分母が追加される。

6.3 S-two / Circle PCS における対応

S-two では circle polynomial を点 Q とその共役点で open し、単変数の $X-z$ に相当する vanishing polynomial で割った quotient を用いる。形は異なるが、「claimed evaluation を差分商へ埋め込み、その低次数性を FRI で検査する」という目的は同じである。[注 2]

注 2 本文の式は古典的単変数 PCS の説明用である。Circle STARK では評価領域、共役点、vanishing polynomial、folding map が異なるため、実装式をそのまま置き換えてはならない。

第 7 章 Query 生成と Merkle decommitment 検証

7.1 query は最後に生成する

検証者は、trace、composition、FRI 各層、final polynomial が transcript へ取り込まれた後に query 位置を生成する。anti-grinding または proof-of-work を採用する場合、nonce の妥当性を確認してから query randomness を導出する。証明者が commitment を変更して有利な query を探すには、grinding を繰り返さなければならない。

同じランダムqueryを、異なるサイズの列とFRI層へ写像する

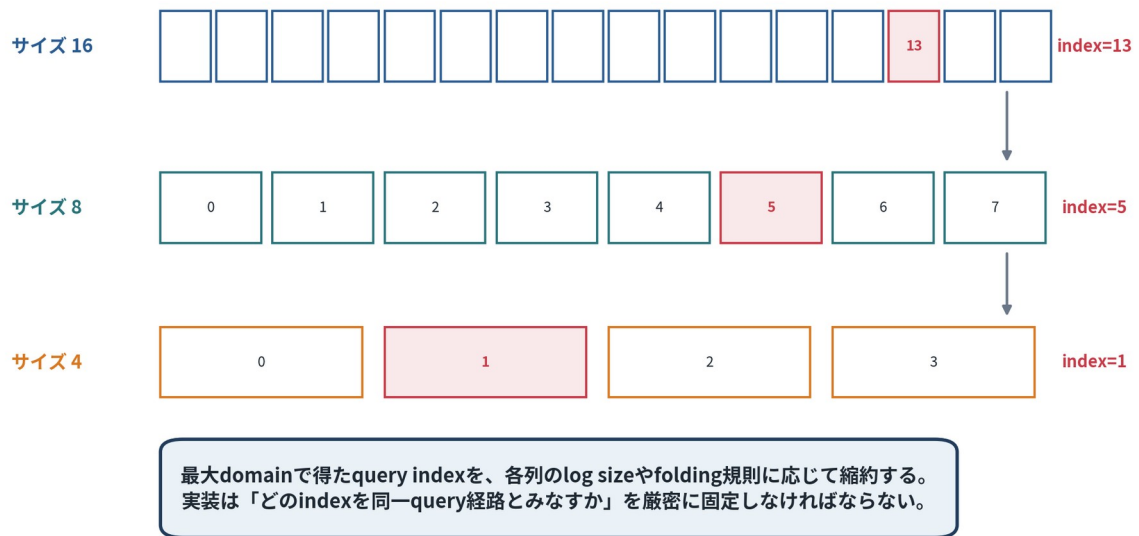


図9 query 位置を異なる column size・FRI layer へ写像する（筆者作成）

7.2 重複 query と domain size

query を生成した結果、同じ index が重複する場合の扱い、最大 domain から小さい domain への写像、bit reversal や coset ordering は仕様で固定する。検証者と証明者が異なる index ordering を使うと、正しい proof でも Merkle path と FRI 経路が一致しない。

7.3 Merkle root を再構成する

検証者は query された leaf value を hash し、proof から受け取った兄弟 hash を左右の順序に従って結合し、root まで反復する。計算した root が先に transcript へ登録した root と一致しなければ拒否する。

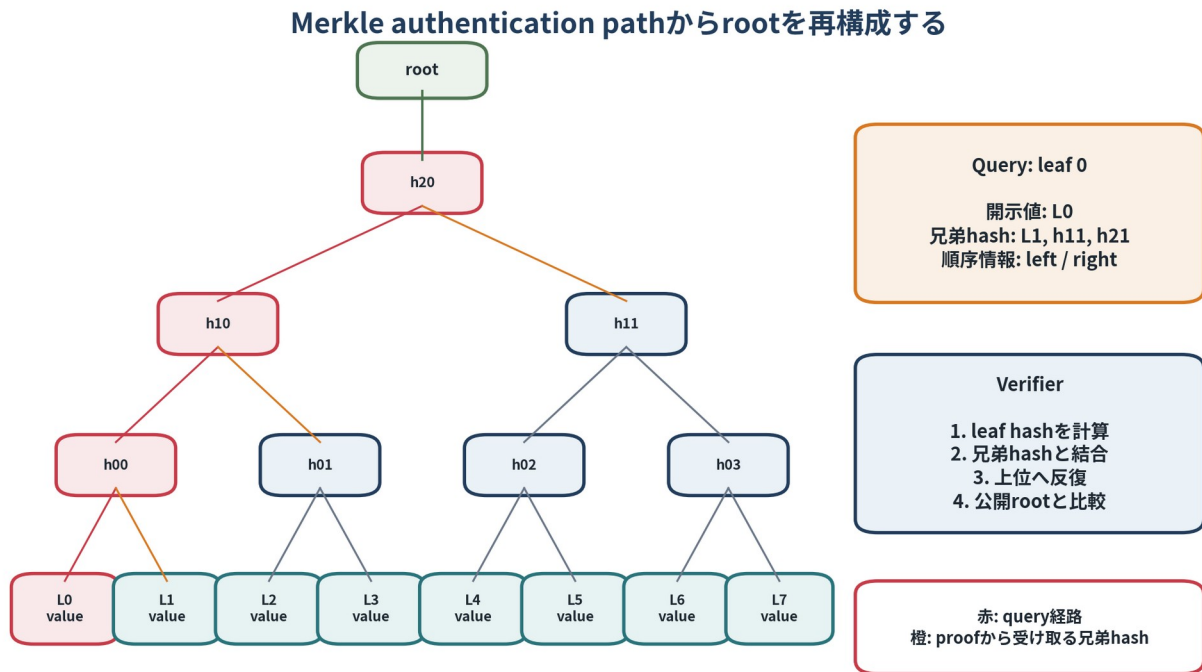


図 10 Merkle authentication path による root 再構成 (筆者作成)

7.4 witness を過不足なく消費する

堅牢な Merkle verifier は、hash witness や column witness が短すぎる場合だけでなく、余分に残る場合も拒否する。余分な要素を黙って無視すると、serialization ambiguity や異なる parser 間の解釈差が生じる。S-two の Merkle verifier も、queried values、hash witness、column witness がすべてちょうど消費されたことと、computed root の一致を確認する。

Merkle 検査	拒否条件	検出する問題
leaf 再計算	field element の encoding が不正	異なる byte 表現、非 canonical 値
path 長	tree 高さと不一致	短すぎる / 長すぎる witness
左右順序	query index と不一致	兄弟 hash の入替え
root 比較	computed root != committed root	後から差し替えた開示値
完全消費	未使用の witness が残る	曖昧な proof 表現、parser 差

第 8 章 FRI verifier の commit phase と query phase

FRI verifier は、最初から query 値だけを見るのではない。まず FRI proof の commitment 構造と次数上限を読み込み、各 layer root を transcript へ mix して folding coefficient を導出する commit

phaseを行う。その後、最終 transcript から生成した query 経路について opening と folding equation を検査する。

FRI verifierは「commit phase」と「query phase」を分けて処理する

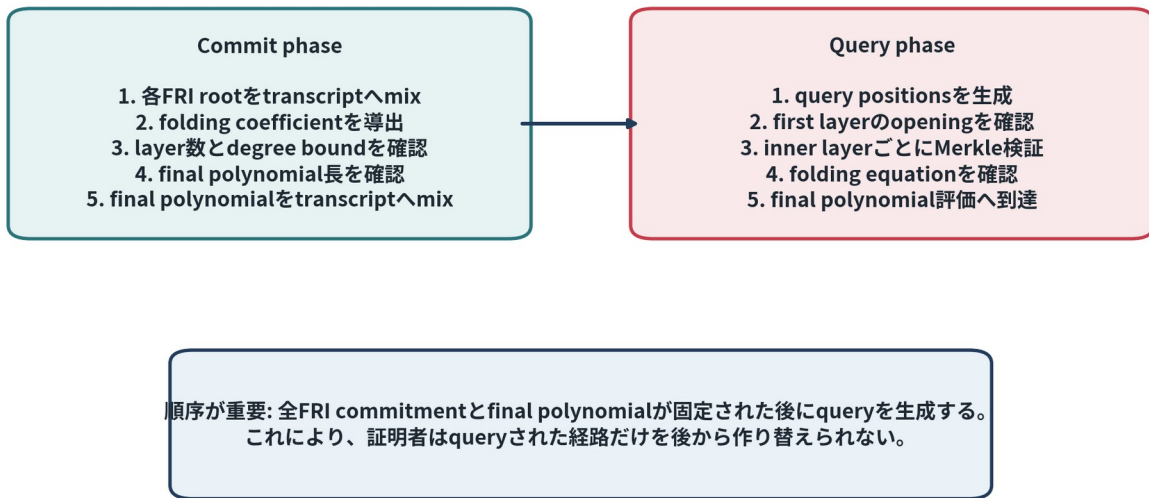


図 11 FRI verifier の二つの phase (筆者作成)

8.1 Commit phase で確認する項目

- first layer と inner layer の commitment root を所定順序で channel へ mix する。
- 各 layer の degree bound と commitment domain size が設定と一致することを確認する。
- fold step を適用した結果、想定された last layer degree へ到達する layer 数であることを確認する。
- final polynomial の係数数が last degree bound 以下であることを確認する。
- final polynomial を channel へ mix し、その後に query を生成する。

8.2 二分 FRI の folding equation

$f(X) = f_{\text{even}}(X^2) + X \cdot f_{\text{odd}}(X^2)$	(7)
$f_{\text{next}}(Y) = f_{\text{even}}(Y) + \beta \cdot f_{\text{odd}}(Y)$	(8)
$f_{\text{next}}(x^2) = (f(x) + f(-x))/2 + \beta \cdot (f(x) - f(-x))/(2x)$	(9)

古典的二分FRI: 対になる二点を次の一値へfoldする

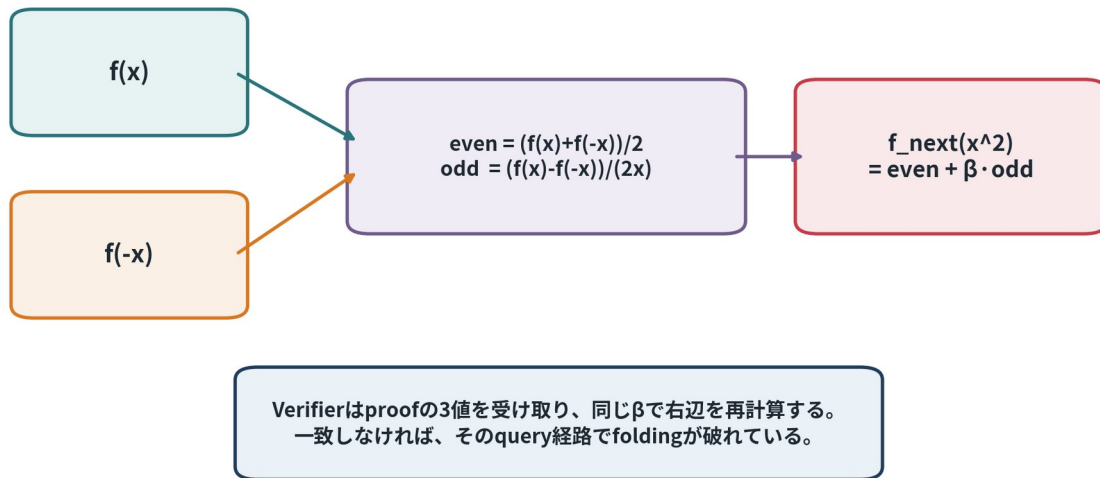


図 12 古典的二分 FRI の folding 検査 (筆者作成)

8.3 Query phase の処理

検証者はfirst layerのquery位置における評価値とMerkle pathを確認し、その評価値から次layerの期待値を計算する。inner layerでも同じ処理を繰り返し、最後に得られた値がfinal polynomialをlast domainの対応点で評価した値と一致することを確認する。

一つのqueryがFRI各層を貫く

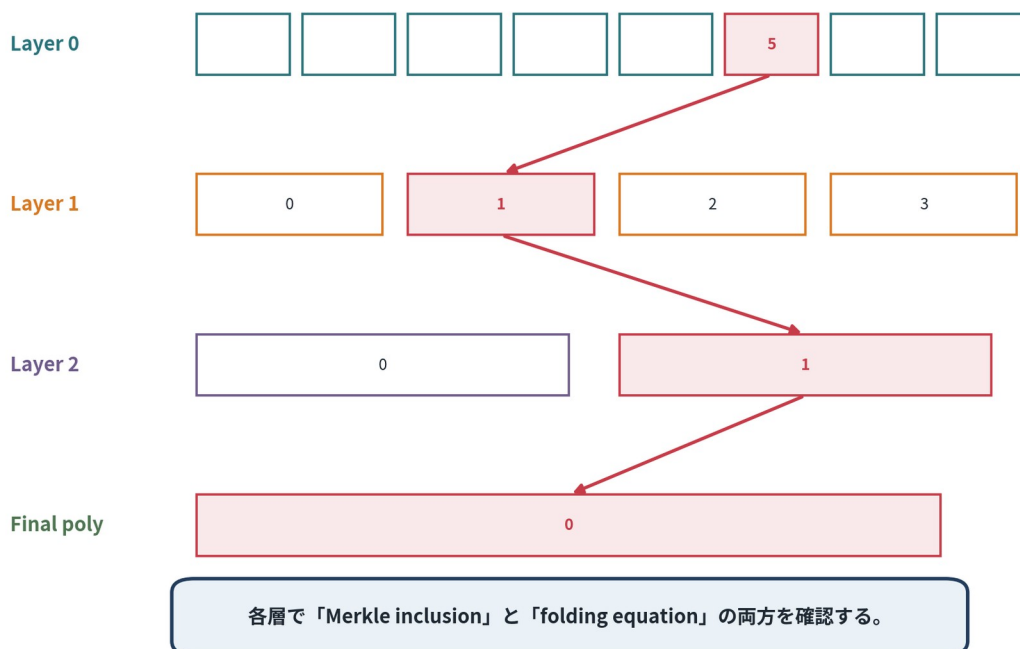


図 13 一つの query が FRI 各層を貫く (筆者作成)

8.4 FRI が直接確認するもの

役割の分離 FRI が直接確認するのは、commit された最終評価表が所定の低次数コードに近いことである。その多項式が正しい計算を表すかは AIR / composition、OODS opening が commitment へ結び付いているかは DEEP / PCS が担う。

8.5 Circle FRI で変わる部分と変わらない部分

Circle FRI では、circle domain、projection、共役点、circle-to-line fold などを使用するため、式 (7)-式(9)をそのまま使用しない。一方、各 layer commitment を先に固定し、ランダムな folding 係数で degree bound を縮小し、少数 query で folding 関係と last polynomial を確認するという検証者の論理は共通である。

第 9 章 最終受理、エラー分類、少数 query でよい理由

9.1 Fail closed で判定する

検証者は、構造、transcript、OODS、Merkle、FRI、公開入力 binding のすべてが成功した場合だけ Valid を返す。ある検査の失敗を警告として扱い、残りが成功したから受理するという多数決は存在しない。

どの段階で、どの不正を拒否するか		
検査段階 入力検査	拒否する代表的な不正 不正な長さ、非canonical field、未知version	概念的エラー InvalidStructure
Transcript	順序違い、domain separation違い、root差替え	ChallengeMismatch
OODS / AIR	sampled valueとcompositionが不一致	OodsNotMatching
Merkle	開示値がcommitment rootに含まれない	RootMismatch / WitnessError
FRI構造	layer数・degree bound・final poly長が不正	InvalidFriStructure
FRI query	folding equationまたはfinal評価が不一致	FriVerificationError
アプリ方針	program ID、nonce、公開入力の結合が不正	PolicyReject

図 14 検証段階と代表的な拒否理由（筆者作成）

9.2 暗号学的 Valid と業務上の許可を分ける

暗号学的検証が成功しても、アプリケーションは nonce の再利用、失効済み credential、期限切れ root、未許可 version、送金上限などを確認する必要がある。Verifier engine の Valid を、そのまま無条件の業務許可とみなしてはならない。

9.3 なぜ少数 query でよいのか

少数 query でよい理由は、query 自体に魔法があるからではない。証明者が先に全評価表へ commit し、AIR が計算意味を制約し、OODS/DEEP が多項式間を結び付け、FRI が評価表全体の低次数近接性を検査するという複数の構造があるため、ランダムな少数経路が全体への統計的検査として機能する。

少数 query で全計算を判断できる「四重のロック」

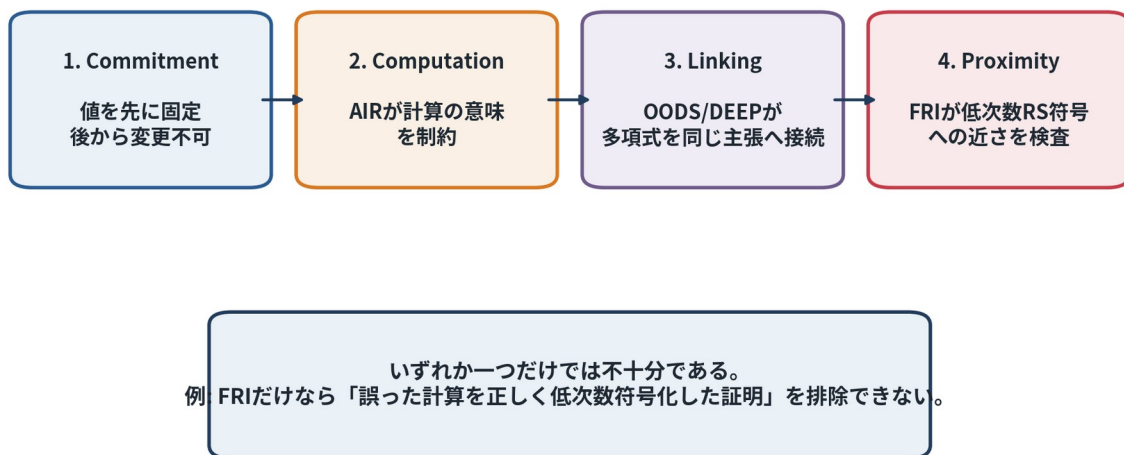


図 15 少数 query を支える四重のロック（筆者作成）

9.4 soundness parameter

不正受理確率は、単純に「query 一回の見逃し率の q 乗」だけで決まるわけではない。code rate、blowup factor、FRI folding、extension field、DEEP の解析、query 数、grinding bits、hash security などが複合する。実装者は、ライブラリが提供する安全性解析と推奨設定を使用し、独自の概算だけで parameter を下げない。

第 10 章 実装・デプロイと S-two verifier への対応

10.1 Off-chain と On-chain の違い

検証者を実サービスへ配置する際の信頼境界

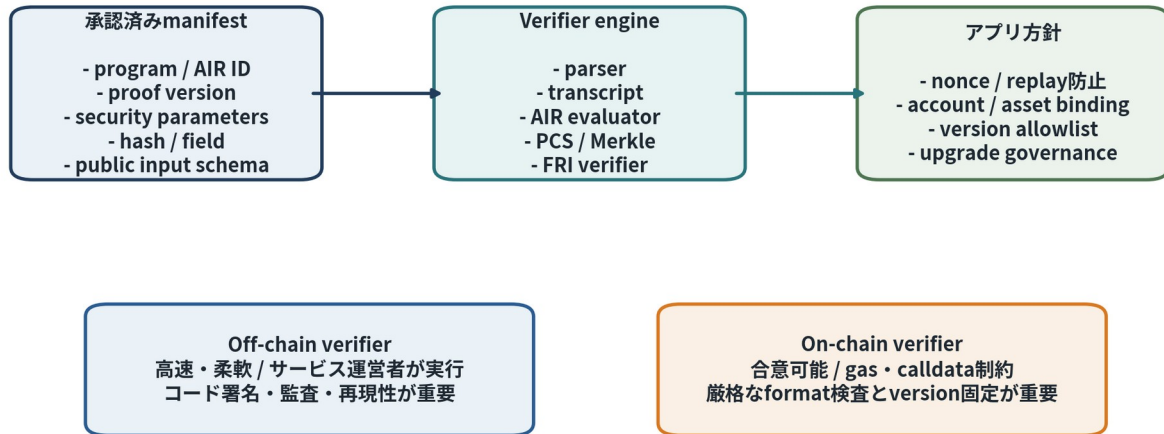


図 16 検証器を実サービスへ配置する際の信頼境界（筆者作成）

Off-chain verifier は CPU・memory を柔軟に使える一方、利用者は実行コード、設定、結果改ざん防止を確認する必要がある。On-chain verifier は複数ノードが同じ判定を共有できるが、gas、calldata、hash primitive、field arithmetic、proof size の制約が大きい。いずれの場合も program / AIR ID と proof version を固定し、upgrade governance を設ける。

10.2 S-two の verify 関数との対応

S-two の公式説明では、verify 関数は AIR components と preprocessed column 数を初期化し、composition polynomial の degree bound を得る。次に random coefficient を channel から引き、composition commitment を登録し、OODS point を導出する。component が必要とする mask points を計算し、proof.sampled_values から composition OODS evaluation を抽出する。検証者が同じ sampled values から再計算した composition evaluation と一致しなければ OodsNotMatching で拒否し、一致した場合に commitment scheme verifier へ進む。

```

composition_oods_eval ==
eval_composition_polynomial_at_point(z,
sampled_values, α)

```

(10)

10.3 S-two の commitment scheme、Merkle、FRI

汎用概念	S-two 内の対応処理	検証内容
Composition OODS	verify()内の再計算	AIR sampled values と composition 評価の一致
PCS opening	commitment_scheme.verify_values	sample points の opening、DEEP、FRI を一括検証
FRI commit phase	FriVerifier::commit	root mix、folding alpha、layer 数、last degree
Query 生成	sample_query_positions	channel から位置を生成し各 log size へ写像
FRI query phase	FriVerifier::decommit	Merkle decommitment と folding equation を検査
Merkle verification	MerkleVerifier::verify	witness 完全消費と computed root 一致

10.4 検証器擬似コード

```

verify(statement, proof, manifest):
  cfg <- load_approved_config(manifest)
  validate_structure_and_canonical_encoding(proof, cfg)
  channel <- init(domain_tag, cfg, statement)
  mix_trace_commitments(channel, proof)
  alpha <- draw_constraint_coefficients(channel)
  mix_composition_commitment(channel, proof)
  z <- draw_oods_point(channel)
  if rebuild_composition_at(z, proof.sampled_values, alpha) != proof.C_z: reject
  initialize_PCS_and_FRI(channel, proof, cfg)
  queries <- draw_queries_after_all_commitments(channel)
  verify_merkle_openings(queries, proof)
  verify_DEEP_and_FRI(queries, proof)
  apply_application_binding(statement, cfg)
  accept

```

(11)

10.5 レビューで重視する点

- parser と transcript が同一の canonical serialization を使用しているか。
- 安全性 parameter を proof 内の値で上書きしていないか。
- OODS sampled_values の列順・offset 順・extension coordinate 順が一致しているか。
- Merkle witness を過不足なく消費し、重複 query を一意に処理しているか。
- FRI layer 数と degree bound の算術で integer overflow が起きないか。
- すべての失敗経路が fail closed であり、panic や例外を Valid へ変換しないか。
- program ID、public input、nonce、version がアプリ層で明示的に結合されているか。

Appendix A F_17 Toy Computation で検証者の処理を追う

本 Appendix では、有限体 F_{17} 上の関係式 $y=(x+1)(x+2)(x+3)$ 、公開値 $y=9$ 、witness $x=2$ を使用する。暗号的な安全性を持たない小規模例であるが、検証者がどの値を再計算するかを手計算で確認できる。

F_17 Toy例: 検証者が追う値の流れ

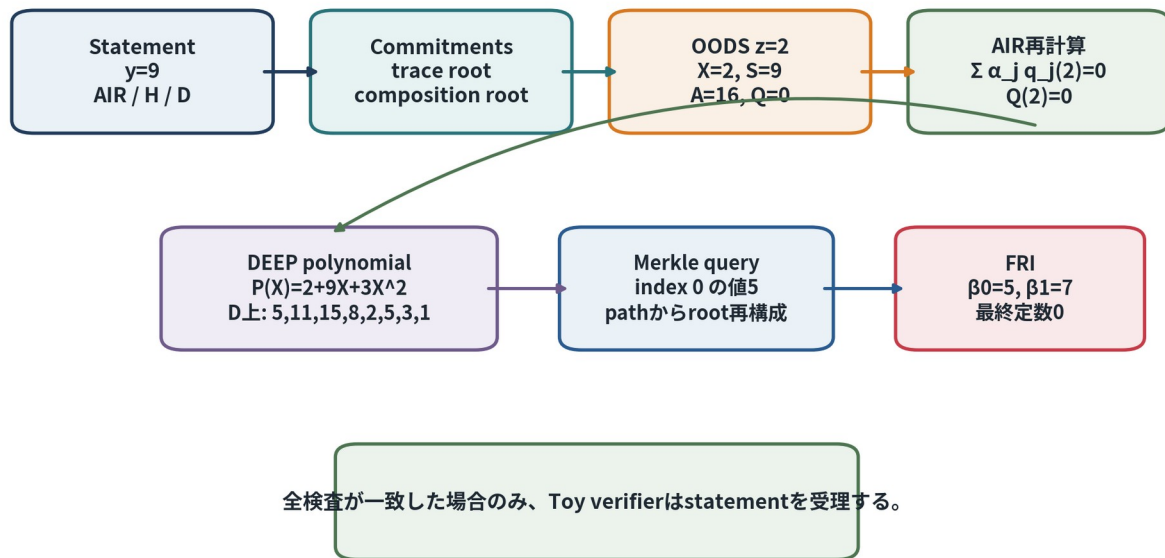


図 17 F_17 Toy verifier が追う値の全体像（筆者作成）

A.1 公開設定と OODS sampled values

項目	値
トレース領域 H	$\{1, 4, 16, 13\}$
LDE 領域 D	$\{3, 6, 12, 7, 14, 11, 5, 10\}$
OODS 点 z	2
制約束ね係数 α	$(2, 3, 5, 7, 11, 13)$
OODS 値	$X(2)=2, S(2)=9, A(2)=16, Q(2)=0$
DEEP 束ね係数 γ	$(1, 2, 3, 4)$
FRI 係数	$\beta_0=5, \beta_1=7$

A.2 検証者は composition OODS 値を再計算する

前資料で導出した制約商を用いる。z=2 における各値は次のとおりである。

制約商	多項式	z=2 での値
q_X	0	0
q_S	4	4
q_A	$11+15X^2+10X^3$	15
q_{A,0}	$16+4X^2$	15
q_{A,3}	$11+14X+4X^2$	4
q_{S,0}	$7+14X+6X^2$	8

$$2 \cdot 0 + 3 \cdot 4 + 5 \cdot 15 + 7 \cdot 15 + 11 \cdot 4 + 13 \cdot 8 = 340 \equiv 0 \pmod{17}$$

(12)

proof が提示した $Q(2)=0$ と一致するため、Toy AIR / composition の OODS 一致検査は成功する。いずれかの sampled value を変更すると、検証者が再計算する右辺が変わり、 $Q(2)$ との不一致として拒否される。

A.3 DEEP polynomial を再構成する

$P_{\text{DEEP}}(X) = \sum \gamma_i \cdot (f_i(X) - f_i(2)) / (X - 2) = 2 + 9X + 3X^2$									(13)
d	3	6	12	7	14	11	5	10	
$P_{\text{DEEP}}(d)$	5	11	15	8	2	5	3	1	

検証者は実際には D 上の全値を受け取らない。query された位置だけを受け取り、Merkle path で commit 済み評価表へ結び付け、FRI で評価表全体の低次数性を検査する。

A.4 Toy Merkle opening

Toy Merkle opening: index 0 の値5からrootを再構成

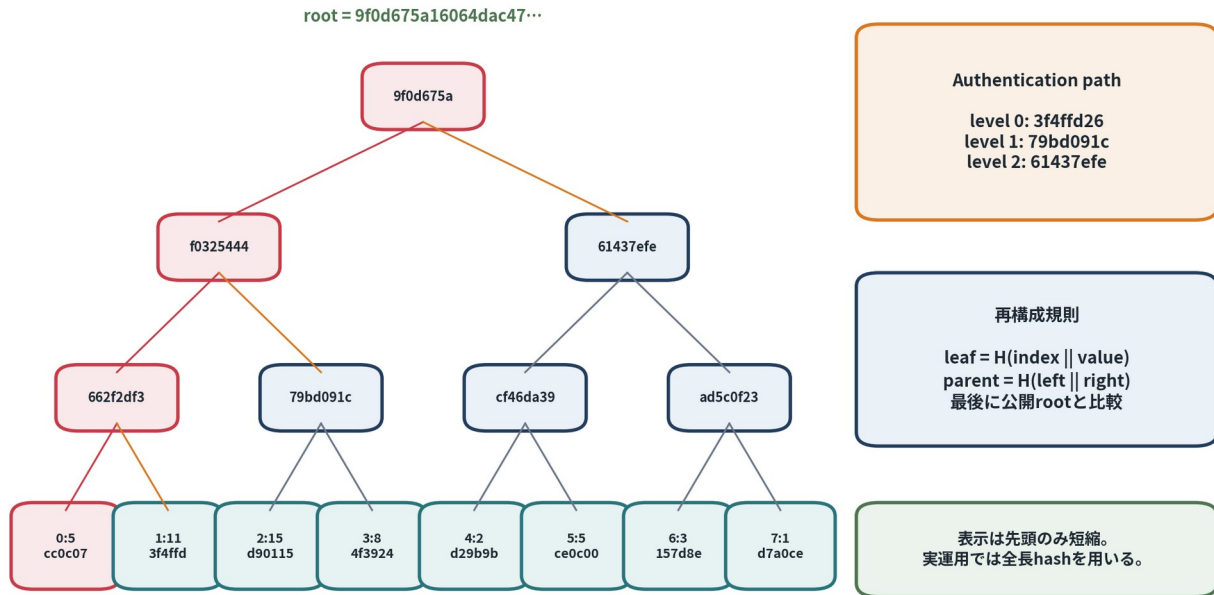
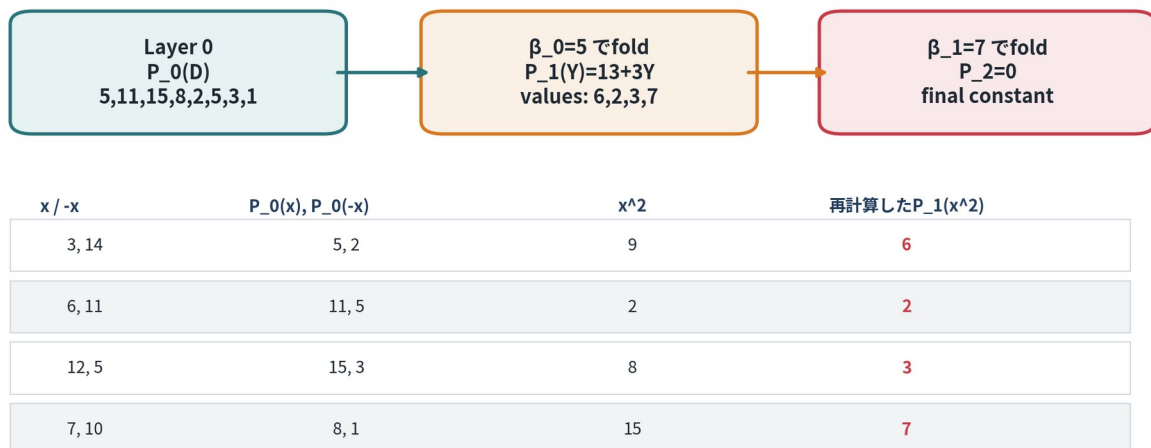


図 18 SHA-256 の短縮表示を用いた Toy Merkle opening (筆者作成)

図 18 では index と値を leaf へ含め、SHA-256 で hash した。表示は読みやすさのため先頭だけに短縮している。検証者は index 0 の値 5 と 3 個の兄弟 hash から root を再計算し、proof 冒頭の commitment root と比較する。

A.5 Toy FRI query 検査

Toy FRI verification: $P_0(X)=2+9X+3X^2$ を定数までfoldする



各行で $P_1(x^2) = (P_0(x) + P_0(-x))/2 + 5 \cdot (P_0(x) - P_0(-x))/(2x)$ を確認する。

図 19 P_{DEEP} に対する Toy FRI folding (筆者作成)

$P_0(X)=2+9X+3X^2$ を偶奇分解すると、 $\text{even}(Y)=2+3Y$ 、 $\text{odd}(Y)=9$ となる。 $\beta_0=5$ により $P_1(Y)=13+3Y$ を得る。さらに $\beta_1=7$ を用いると $P_2=13+7\cdot 3=34\equiv 0$ となる。検証者は query された各ペアについて fold 式を再計算し、最後に final polynomial の定数 0 と一致することを確認する。

Toy 例の限界 Toy 例では query 数が少なく有限体も小さいため、安全性を示さない。目的は、OODS 再計算、DEEP 差分商、Merkle inclusion、FRI folding が一つの検証経路として接続されることを確認することである。

Appendix B 検証器設計・レビュー用チェックリスト

分類	確認事項
Statement binding	program / AIR ID、public input、nonce、chain / domain ID を transcript へ含めるか
Configuration	field、hash、degree bounds、FRI 設定を proof 外の承認済み manifest から固定するか
Parser	長さ上限、canonical encoding、unknown version、整数 overflow を拒否するか
Transcript	domain separation、mix 順、challenge sampling をテストベクトルで固定するか
OODS	禁止点を避け、必要な mask point と sampled value 構造を照合するか
Composition	AIR 再計算値と proof の composition OODS 値を厳密に比較するか
PCS / DEEP	claimed opening を差分商へ埋め込み、FRI 入力へ正しく束ねるか
Merkle	左右順、path 長、root 一致、witness 完全消費を確認するか
FRI	layer 数、degree bound、fold coefficient、final polynomial を確認するか
Queries	全 commitment 後に生成し、重複と異なる log size を仕様どおり処理するか

分類	確認事項
Errors	すべての異常を fail closed で拒否し、panic / exception を安全に扱うか
Deployment	verifier code hash、manifest hash、upgrade governance、旧 version 失効を管理するか

Appendix C 用語集

用語	本資料における意味
Proof object	commitments、OODS 値、FRI proof、query openings 等を含む serialized 証明構造
Manifest	program ID、field、hash、degree bounds、FRI 設定等を固定する承認済み設定
Fiat-Shamir channel	transcript を hash し、検証者 challenge を非対話的に再現する状態機械
Sampled values	OODS 点と必要 offset における trace / composition 等の claimed evaluation
Composition polynomial	複数 AIR 制約商をランダム係数で束ねた多項式
OODS	通常の評価領域外から challenge point を選び、多項式評価を要求する処理
DEEP quotient	claimed evaluation を分子へ差し込み、vanishing factor で割った opening 用多項式
Merkle decommitment	query 値が commit 済み評価表に含まれることを示す兄弟 hash 等
FRI commit phase	layer roots、fold coefficient、degree bounds、final polynomial を固定する段階
FRI query phase	少数経路で Merkle opening、folding equation、final 評価を確認する段階
Fail closed	不明・不一致・例外が生じた場合に必ず拒否側へ倒す設計

参考文献

- [1] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Scalable, Transparent, and Post-Quantum Secure Computational Integrity,” IACR ePrint 2018/046, 2018.
- [2] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Fast Reed-Solomon Interactive Oracle Proofs of Proximity,” ICALP 2018, LIPIcs 107, Article 14, 2018. DOI: 10.4230/LIPIcs.ICALP.2018.14.
- [3] E. Ben-Sasson, L. Goldberg, S. Kopparty, and S. Saraf, “DEEP-FRI: Sampling Outside the Box Improves Soundness,” ITCS 2020, LIPIcs 151, Article 5; IACR ePrint 2019/336.
- [4] Starknet Documentation, “STARK Verifier,” S-two Book, verify function, accessed 2026-08-13.
- [5] Starknet Documentation, “FRI Verifier,” S-two Book, FriVerifier commit, query generation, and decommit phases, accessed 2026-08-13.
- [6] Starknet Documentation, “Merkle Verifier,” S-two Book, MerkleVerifier::verify, accessed 2026-08-13.
- [7] Starknet Documentation, “Polynomial Commitment Scheme - Technical Overview,” S-two Book, OODS, quotient openings, and proof of work, accessed 2026-08-13.
- [8] Starknet Documentation, “STARK Prover,” S-two Book, composition polynomial, OODS values, and commitment scheme, accessed 2026-08-13.

注記の表示方法

難しい概念に関する注記は、参照位置を示す [注番号] と、該当箇所付近の灰色欄で表示している。本文の古典的 FRI 式と S-two / Circle FRI の実装式は区別して読む必要がある。